



Zebebgna

“Guard” — Defensive verification of Ethiopian bank receipts

Version 0.1.0

Cybersecurity toolkit · CLI + Web UI + Telegram Bot

CBE · Dashen · Awash · BOA · Zemen · Telebirr

<https://github.com/natahanjr/zebebgna>

Table of Contents

1. About Zebebgna
2. Features
3. Scope & Ethics
4. Architecture
5. Installation
6. Command-Line Interface (CLI)
 - 6.1 Verify a receipt · 6.2 Audit an endpoint · 6.3 Batch, text & history · 6.4 Threat DB · 6.5 Operational tooling · 6.6 Sample output
7. Python API
 - 7.1 verify_receipt · 7.2 audit_receipt_url · 7.3 Vision / LLM · 7.4 VerificationReport & Finding
8. Web UI for Non-Technical Users
 - 8.1 Running · 8.2 Hardening for shared use · 8.3 History & shareable report links
9. Security Checks Reference
10. Risk Scoring & Verdicts
11. Threat Fusion Engine
12. Community Phishing Database & Feedback Loop
13. Telegram Bot
14. Supported Banks & Reference Formats
15. Testing
16. Project Structure
17. Limitations & Deployment Notes
18. License

1. About Zebebgna

A receipt is only as trustworthy as its source. **Zebebgna** (Amharic for *guard*) is a cybersecurity toolkit that keeps fake and tampered receipts out of your hands. It extracts receipt data from six major Ethiopian banks and mobile-money services — **CBE, Dashen, Awash, BOA, Zemen, and Telebirr** — then puts that data on trial:

- It checks the serving website's security posture (TLS, certificates, security headers).
- It flags phishing lookalikes and suspicious URLs before they are trusted.
- It reconciles amounts, validates reference numbers, and verifies transaction status.
- It produces a clear, color-coded verdict: **Good**, **Needs review**, or **Problem found**.

Zebebgna is the defensive counterpart to naive receipt-scraping libraries — it never trusts the receipt it fetches, and it never disables security checks to make a connection work.

CLI

For developers and security teams. One command to verify a receipt or audit an endpoint, with machine-readable JSON output.

Web UI

For non-technical users. A simple browser form: choose the bank, paste the receipt link, get a color-coded verdict with a list of checks.

2. Features

Feature	Description
 Strict TLS everywhere	Every fetch verifies the certificate chain. Plain-HTTP URLs are refused; the insecure <code>verify_ssl=False</code> shortcuts common in scrapers are never used.
 Phishing detection	Flags raw-IP hosts, lookalike domains (typosquatting against known bank domains), punycode hosts, URL shorteners, and non-standard ports.
 TLS certificate audit	Chain validation, hostname coverage (SAN), expiry warnings, and issuer reporting.
 Security headers audit	Checks HSTS (incl. <code>includeSubDomains</code>), CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.
 Authenticity verification	Cross-field amount reconciliation (e.g. transferred + commission + VAT $\approx$ total debited), per-bank reference-number format checks, transaction-status validation, completeness checks.
 Risk scoring	Every check yields a severity-tagged finding; the report summarizes a 0–100 score and a PASS / REVIEW / FAIL verdict.
 Threat fusion	A deterministic rule engine (<code>zebebgna/fusion.py</code>) correlates weak signals across check groups into attack scenarios (lookalike phishing infrastructure, forged receipts, weak hardening) with a fused 0–100 risk score and LOW / MEDIUM / HIGH / CRITICAL level.
 History & feedback loop	Every check can be stored in local SQLite; users mark verdicts correct/wrong and the per-domain feedback nudges future risk scores. A community phishing-domain database (<code>threatdb</code>) raises an error-level signal for reported domains.
 Vision / OCR & AI review	Receipt text can be pasted or uploaded as a PDF/screenshot; <code>scan_fields()</code> extracts structured fields (English and Amharic amounts, Ge'ez numerals, Ethiopian calendar dates). An optional OpenAI-compatible review (<code>ZEBEBGNA_LLM_API_KEY</code>) adds a genuine/suspicious verdict; failures degrade silently.
 Telegram bot	Optional bot (<code>python -m zebebgna.bot</code>): <code>/verify</code> , <code>/verifytext</code> , and photo/PDF uploads, all returning the same verdicts.
 Shareable reports	The web UI offers a <code>/share/<check_id></code> archived-report link for every stored check.
 Operational tooling	<code>zebebgna watch</code> re-verifies on a schedule, <code>zebebgna backup export import</code> dumps/restores the database, and <code>zebebgna config show</code> displays the effective configuration.

⚡ High performance

Connection pooling via `requests.Session` ; multithreaded PDF parsing;
cached Chrome driver for BOA receipts.

3. Scope & Ethics

This tool is built for **defensive security only**:

- Verify receipts you are entitled to see (your own transactions, authorized reconciliation).
- Detect phishing lookalike domains and tampered receipts before they are trusted.
- Report transport weaknesses (expired/invalid TLS certs, missing security headers, insecure URLs) to the operators who control them.

Not a tool for bypassing access controls, enumerating accounts, or attacking bank infrastructure.
Unauthorized probing of third-party systems is illegal in most jurisdictions — audit only endpoints you own or are authorized to test.

4. Architecture



Component	Responsibility
<code>zebebgna.fetch</code>	<code>SecureFetcher</code> — HTTPS-only fetching with strict TLS verification, timeouts, bounded redirects, and connection pooling. Refuses non-HTTPS URLs (<code>InsecureURLError</code>).
<code>zebebgna.extractors</code>	Per-bank parsing: PDF extraction (CBE, Dashen, Zemen) via <code>pdfplumber</code> ; HTML tables (Awash, Telebirr) via <code>BeautifulSoup</code> ; rendered pages (BOA) via headless Chrome.
<code>zebebgna.verifiers</code>	Four independent check groups: <code>phishing</code> , <code>tls</code> , <code>headers</code> , <code>integrity</code> .
<code>zebebgna.report</code>	Data model — <code>Finding</code> and <code>VerificationReport</code> with scoring and verdict logic.
<code>webapp.py</code>	Flask application exposing the same pipeline through a browser form for non-technical users.

5. Installation

Requires Python 3.8+. From the project directory:

```

pip install -e .           # core (CLI + library)
pip install -e ".[web]"    # + Flask web UI
pip install -e ".[bot]"    # + Telegram bot
pip install -e ".[dev]"    # + pytest

```

For BOA receipts, a **ChromeDriver** matching your installed Chrome is required (the extractor drives a headless Chrome browser because BOA receipts render client-side).

6. Command-Line Interface (CLI)

6.1 Verify a receipt

```

zebebgna verify cbe      "https://apps.cbe.com.et:100/?id=FT25211G11JQ21827223"
zebebgna verify dashen   "https://receipt.dashensuperapp.com/receipt/387ETAP2522000WK"
zebebgna verify tele     CHQ0FJ4030
zebebgna verify cbe      ./receipt-screenshot.png    # local PDF/image file

```

Telebirr accepts a bare receipt ID; all other banks require the full receipt URL. A local PDF or image path runs OCR-based extraction instead of fetching. If the module is not on your PATH, run `python -m`

zebebgna.cli ... instead.

6.2 Audit an endpoint

Runs only the transport/URL-level checks (phishing, TLS, headers) without extracting receipt data:

```
zebebgna audit "https://receipt.dashensuperapp.com/receipt/387ETAP2522000WK"
```

6.3 Batch, pasted text & history

```
zebebgna batch cbe --file urls.txt           # JSON array output
zebebgna verify-text cbe "Transaction ID: ... Amount: 1,000.00"
zebebgna history --limit 20
zebebgna history --clear
zebebgna feedback 42 --correct | --wrong [--report-phish "reason"]
```

All verification runs are stored in the local history database unless `--no-save` is given; `feedback` marks a past verdict and nudges future risk scores for the same domain, and `--report-phish` also adds the domain to the community phishing database.

6.4 Community phishing database

```
zebebgna threatdb add "evil-bank-eth.com" "seen in Telegram"
zebebgna threatdb list
zebebgna threatdb remove "evil-bank-eth.com"
```

6.5 Operational tooling

```
zebebgna watch cbe "https://apps.cbe.com.et:100/?id=..." --every 120 --count 10
zebebgna backup export backup.sql
zebebgna backup import backup.sql          # replaces local data with the dump
zebebgna config show
```


6.6 Sample output

```
{
  "url": "https://receipt.dashensuperapp.com/receipt/387ETAP2522000WK",
  "bank": "dashen",
  "score": 74,
  "status": "REVIEW",
  "extracted_data": { "...": "..." },
  "findings": [
    {"severity": "info", "category": "tls", "message": "Certificate valid until ..."},
    {"severity": "warn", "category": "headers", "message": "Missing security header: Content-Security-Policy"},
    {"severity": "error", "category": "integrity", "message": "Dashen transaction total mismatch: ..."}
  ],
  "threat": {
    "risk_level": "MEDIUM",
    "risk_score": 50,
    "scenario": "Receipt amounts do not reconcile",
    "correlations": [{"rule_id": "amount_conflict", "severity": "error", "...": "..."}]
  }
}
SCORE: 74/100   STATUS: REVIEW
THREAT: MEDIUM (50/100)
SCENARIO: Receipt amounts do not reconcile
```

7. Python API

7.1 `verify_receipt(bank, url_or_id)`

Full pipeline: extract → verify. Returns a `VerificationReport` .

```
from zebebgna import verify_receipt

report = verify_receipt("cbe", "https://apps.cbe.com.et:100/?id=FT25211G11JQ21827223")
print(report.score)      # 0-100
print(report.status)     # PASS | REVIEW | FAIL
for f in report.findings:
    print(f.severity.upper(), "-", f.category, ":", f.message)
```

7.2 `audit_receipt_url(url)`

Transport-only audit (no extraction):

```
from zebebgna import audit_receipt_url
report = audit_receipt_url("https://example-receipt-host/abc123")
```

7.3 Vision, OCR & optional AI review

```
from zebebgna import verify_file, verify_extracted_data
from zebebgna.vision import scan_fields

report = verify_file("cbe", "receipt.png") # PDF/image
report = verify_extracted_data("tele", scan_fields("tele", text), source="pasted-text")
print(report.ai_review.verdict) # genuine | suspicious | unclear (if
ZEBEBGNA_LLM_API_KEY set)
print(report.threat.risk_level) # LOW | MEDIUM | HIGH | CRITICAL
```

7.4 VerificationReport & Finding

Attribute	Description
<code>report.url</code>	The URL that was checked.
<code>report.bank</code>	Bank key (<code>cbe</code> , <code>dashen</code> , <code>awash</code> , <code>boa</code> , <code>zemen</code> , <code>tele</code>), or <code>None</code> for audits.
<code>report.data</code>	Extracted receipt fields as a dict.
<code>report.findings</code>	List of <code>Finding</code> objects.
<code>report.score</code>	0–100 security score.
<code>report.status</code>	PASS, REVIEW, or FAIL.
<code>report.threat</code>	<code>ThreatAssessment</code> from the fusion engine (risk score, level, scenario, correlations), or <code>None</code> .
<code>report.ai_review</code>	<code>AIVerdict</code> (genuine / suspicious / unclear) when the optional LLM review ran.
<code>report.to_dict()</code>	Full JSON-serializable report.

Each `Finding` has `severity` (info / warn / error / critical), `category` (tls / url / phishing / transport / headers / integrity / fetch), and `message` .

8. Web UI for Non-Technical Users

8.1 Running

```
pip install -e ".[web]"
python webapp.py
# open http://127.0.0.1:5000
```

The user picks the bank, pastes the receipt link (or Telebirr ID), and clicks **Verify receipt**. The result page shows:

- A color-coded verdict: **GOOD** **NEEDS REVIEW** **PROBLEM FOUND**, plus the 0–100 score.
- All performed checks with severity badges and plain-language messages.
- The extracted receipt details in a readable table.

8.2 Hardening for shared use

The web app is a server that fetches URLs on behalf of its users, so it must be locked down before exposing it beyond your own machine:

- **HTTPS-only**: plain-HTTP URLs are refused by the secure fetcher.
- **Host allowlist (SSRF guard)**: restrict which hosts the server may fetch:

```
zebebgna_ALLOWED_HOSTS="apps.cbe.com.et,transactioninfo.ethiotelecom.et" python webapp.py
```

- **Authentication**: put the app behind a reverse proxy with a login (e.g. basic auth or SSO).
- **Binding**: the default binds to `127.0.0.1` only — keep it that way unless you truly need remote access.

8.3 History & shareable report links

The web UI keeps a history of every check (`/history`) with thumbs-up/down feedback buttons, and offers an archived, read-only **share link** (`/share/<check_id>` — the "Share" chip on a fresh verdict page). Shared pages show the same report without the history/admin chrome, so a verified receipt can be referenced later without exposing the whole history.

9. Security Checks Reference

9.1 URL / phishing checks (`phishing.py`)

Check	Severity	Condition
Insecure scheme	critical	URL is not <code>https</code> .
Raw IP host	critical	Host is a literal IP address (e.g. <code>192.168.1.10</code>).
Lookalike domain	warn	Registered domain $\geq$ 72% similar to a known bank domain (typosquatting).
Punycode host	warn	Host uses IDN encoding (<code>xn--</code>).
URL shortener	warn	Host is a known shortener (bit.ly, t.co, tinyurl.com, ...).
Non-standard port	warn	HTTPS on a port other than 443.

9.2 TLS checks (`tls.py`)

Check	Severity	Condition
Chain verification	critical	Certificate cannot be validated against trusted roots.
Hostname coverage	critical	Certificate SAN does not cover the hostname.
Expired certificate	critical	Current date is past <code>notAfter</code> .
Near expiry	warn	Certificate expires within 30 days.
Connection failure	error	No TLS session could be established.
Issuer / protocol	info	Informational: issuer and TLS version.

9.3 Security headers (`headers.py`)

Header	Severity if missing	Purpose
Strict-Transport-Security	warn	Forces HTTPS in browsers; blocks SSL-stripping.
Content-Security-Policy	warn	Limits what the page can load; contains XSS damage.
X-Frame-Options	warn	Prevents clickjacking via iframe embedding.
X-Content-Type-Options	warn	Disables MIME sniffing.
Referrer-Policy	info	Controls referrer leakage.
Permissions-Policy	info	Restricts browser feature access.

Important: missing headers are hardening gaps, not proven exploits. A receipt site without these headers can still be completely legitimate — the verdict wording accounts for this via scoring.

9.4 Integrity checks (`integrity.py`)

Check	Severity	Description
Transaction status	critical	Status other than SUCCESS / COMPLETED / PAID / DONE.
Amount reconciliation	error	Components (transferred + commission + VAT) do not sum to the stated total (± 1 ETB). Per-bank for CBE, Dashen, Zemen.
Reference format	error	Reference number does not match the bank's expected format (e.g. CBE references must start with FT).
Missing fields	warn	Required receipt fields (amount, date, reference, ...) are absent.
Non-positive amount	error	Telebirr total paid is zero or negative.

10. Risk Scoring & Verdicts

Each finding carries a severity penalty subtracted from 100:

Severity	Penalty	Example
info	2	Certificate issuer, missing optional headers
warn	10	Missing HSTS/CSP, near-expiry cert, lookalike domain
error	20	Amount mismatch, bad reference, no connection
critical	40	Expired/invalid cert, phishing IP host, failed status

Score	Status	Meaning
≥ 85	PASS	No material findings; receipt can be trusted.
55 – 84	REVIEW	Weaknesses present; review notes before relying on the receipt.
< 55	FAIL	Significant problems; do not trust without resolution.

Any **critical** finding forces FAIL regardless of score.

11. Threat Fusion Engine

`zebebgna/fusion.py` combines weak, individually inconclusive signals across check groups into attack scenarios. A deterministic rule table fires when its predicate matches the report's signals; each fired rule becomes a `Correlation` with a severity, and the fused **risk score** is weighted by the fired correlations' severities (no external feeds — fully reproducible).

Scenario family	Fires when
Lookalike phishing infrastructure	Registered domain is suspicious / community-reported and transport hardening is weak.
Forged / tampered receipt	Integrity errors (amount conflicts, bad references) co-occur with missing security headers or weak TLS.
Weak hardening	Only transport-policy weaknesses (headers/TLS) without other signals.
Community report	The registered domain is in the local <code>threat_domains</code> table (error-level signal).

Per-domain user feedback adjusts the risk score deterministically: **-10** when ≥ 3 rejected verdicts outweigh confirmations, **+5** in the opposite case (capped, and never lifts an unreadable receipt off 0). The adjustment is reported as `feedback_adjustment` on the assessment.

12. Community Phishing Database & Feedback Loop

Reported domains live in the local SQLite database (table `threat_domains`):

```
zebebgna threatdb add "evil-bank-eth.com" "fake receipt link on Telegram"
zebebgna threatdb list
zebebgna feedback 42 --wrong --report-phish "confirmed scam"
```

Once a domain is listed, every later check of that domain gets an error-level `community_reported` correlation in fusion, so the community's experience protects every user on the same machine. The web UI and Telegram bot both benefit automatically; there is no network upload — the registry is local by design.

13. Telegram Bot

```
pip install -e ".[bot]"
set ZEBEBGNA_TELEGRAM_TOKEN=your_token
python -m zebebgna.bot
```

- `/verify <bank> <link-or-id>` — verify a receipt link or Telebirr ID.
- `/verifytext <bank> <text>` — verify pasted receipt text.
- Photo / PDF upload with the bank name in the caption — OCR-based verification.
- Replies carry the score, verdict, threat level/scenario, findings, and the AI review (if enabled).

The reply/parsing logic (`parse_verify_command` , `format_verdict`) is pure and unit-tested without a token; bank aliases (cbe/commercial, boa/abyssinia, tele/telebirr) are accepted.

14. Supported Banks & Reference Formats

Bank	Type	Reference pattern	Extraction method
CBE	Bank	FT + 6–20 alphanumerics	PDF (pdfplumber, threaded)
Dashen	Bank	8–24 alphanumerics	PDF (pdfplumber, threaded)
Awash	Bank	6–30 alphanumerics/dashes	HTML table
BOA	Bank	6–40 alphanumerics/dashes	Headless Chrome
Zemen	Bank	6–30 alphanumerics	PDF (pdfplumber, threaded)
Telebirr	Mobile money	8–12 uppercase alphanumerics	HTML page

CBE additionally supports the helper `extract_cbe_receipt_info_from_ft(ft_number, account)`, which builds the receipt URL from an FT number plus the last 8 digits of the account.

15. Testing

```
pip install -e ".[dev]"
pytest tests/
```

The suite (174 tests) covers URL/phishing heuristics, integrity math, report scoring, the SSRF guard (private hosts, redirect re-validation, allowlists), secure-fetch refusals, the TLS failure path, the full verification pipeline (network mocked), the web UI form/report/allowlist behavior, history & feedback, the fusion rule table, the threat-domain database and CLI commands, the vision/date/amount-words parsers, the optional LLM path, the Telegram bot's pure logic, and the `watch / backup / config` tooling. Live-network extraction tests are deliberately not part of the default suite.

16. Project Structure

```
zebebgna/
+-- pyproject.toml      # package metadata, deps, CLI entry point
+-- README.md          # project documentation
+-- webapp.py           # Flask web UI for non-technical users
+-- zebebgna/
|   +-- __init__.py     # public API: verify_receipt, audit_receipt_url,
verify_file, ...
|   +-- cli.py          # zebebgna verify / audit / batch / watch / backup / config
...
|   +-- bot.py          # optional Telegram bot
|   +-- fetch.py        # SecureFetcher (HTTPS-only, TLS-verified)
|   +-- report.py       # Finding, VerificationReport, scoring
|   +-- fusion.py       # threat-fusion rule engine + ThreatAssessment
|   +-- history.py      # SQLite checks + threat domains + feedback + backup
|   +-- vision.py       # scan_fields() field extraction (amounts, dates,
references)
|   +-- words.py dates.py # Amharic/Ge'ez amount words, Ethiopian calendar dates
|   +-- llm.py          # optional OpenAI-compatible AI review (AIVerdict)
|   +-- templates/     # web UI pages (index, report, history)
|   +-- extractors/    # per-bank parsing
|       | +-- cbe.py dashen.py awash.py boa.py zemen.py tele.py
|   +-- verifiers/     # security check groups
|       +-- phishing.py tls.py headers.py integrity.py fingerprint.py
+-- tests/             # 174 tests (verifiers + SSRF + fusion + web + bot + ops)
+-- docs/              # this documentation
```

17. Limitations & Deployment Notes

- **Regional restrictions:** Telebirr often blocks or times out requests from foreign IPs (403 / ERR_FAILED / DNS failures). It works best from servers inside Ethiopia; VPS providers abroad (AWS, Hetzner, ...) are typically affected.
- **BOA requires ChromeDriver:** the BOA extractor needs a headless Chrome browser and caches the driver session for performance.
- **Regex-based parsing:** extraction relies on each bank's receipt layout. If a bank changes its format, patterns in the relevant extractor must be updated.
- **OCR quality:** pasted text and photographed receipts depend on the text quality; `verify-text` handles common formats but a poor crop can yield missing fields (reported as warnings, never as silent pass).
- **Optional AI review:** the LLM review is advisory and disabled when `ZEBEBGNA_LLM_API_KEY` is not set; platform or quota failures degrade silently to a deterministic report.

- **Community database is local:** the `threat_domains` registry never syncs over the network; if you want shared intelligence, export/import the database (`zebebgna backup`) between machines yourself.
- **Passive verification only:** Zebebgna checks what a receipt and its endpoint *say*; it cannot prove the bank's ledger state. It detects tampering, phishing, and weak transport — it is not a replacement for bank statements.

18. License

MIT License — see <https://opensource.org/licenses/MIT>.